# CS 432: Databases

## FixIIT – Maintenance Portal

Assignment 3 - Module B


**Submitted by: Team Codebase**

Abhitej Singh Bhullar (23110009)
Shiv Patel (23110302)
Shivansh Shukla (23110303)
Soham Shrivastava (23110315)
Arin Mehta (23110038)


**Github Link:**
https://github.com/ArinMehta/FixIIT

**Video Link:**
https://drive.google.com/drive/folders/15rf1huTtG03DJ6u9VV1DXeDZxEMGLlDp

# Contents


**1 Project Objective** **2**

**2 System Overview** **2**

**3 Testing Architecture** **2**
- 3.1 Custom Script Workflow . . . 3
- 3.2 What Each Workload Tests . . . 3

**4 Experimental Setup** **3**
- 4.1 Metrics Collected . . . 4

**5 Experimental Results** **4**

**6 Detailed Workload Analysis** **5**
- 6.1 Concurrent Read Testing . . . 5
- 6.2 Concurrent Write Testing . . . 5
- 6.3 Race Condition Testing . . . 5
- 6.4 Mixed Workload Testing . . . 5
- 6.5 Constraint Violation and Failure Injection . . . 6
- 6.6 Failure During Execution . . . 6

**7 ACID Analysis** **7**
- 7.1 Atomicity . . . 7
- 7.2 Consistency . . . 7
- 7.3 Isolation . . . 7
- 7.4 Durability . . . 7

**8 Authentication Overhead Under Load** **7**

**9 Discussion** **8**

**10 Conclusion** **8**

# 1 Project Objective

Module B shifts the focus from building FixIIT to stress-testing it. In earlier phases we concentrated on schema design, authentication, RBAC, and getting the Flask endpoints working correctly. Here, the question is whether those endpoints hold up once multiple users hit them simultaneously.

Specifically, we wanted to know: does the backend stay correct under concurrent reads and writes? What happens when two users try to update the same ticket at the same time? Does the system properly reject bad input even under load? And if something fails mid-operation, does the database end up in a clean state?

We tested five scenarios to answer these:

- concurrent reads from multiple authenticated clients,
- concurrent writes (ticket creation) from multiple clients,
- multiple clients updating the same ticket simultaneously,
- a realistic mixed read/write workload,
- deliberately malformed requests to test constraint enforcement and rollback.

# 2 System Overview

The backend under test is the Flask application from Assignment 2 Module B, backed by MySQL with the InnoDB storage engine. All testing went through the actual HTTP API — we did not touch the database directly. This was intentional: we wanted the test results to reflect what real users experience, including the overhead of JWT validation, request parsing, and RBAC checks.

The endpoints we exercised most heavily were:

- `/login` — for obtaining auth tokens,
- `/isAuth` — session validation,
- `/tickets` — listing and creating tickets,
- `/tickets/<id>` — updating a specific ticket (used in race condition tests),
- `/portfolio/me` — authenticated profile access.

# 3 Testing Architecture

We used two tools. The primary one is a custom multi-threaded Python script that we wrote for this module. It authenticates multiple clients, fans out requests using a thread pool, collects per-request timings, and writes a summary CSV at the end. We also set up a Locust configuration for broader virtual-user simulation, but the numbers reported here come from the Python script.

### 3.1 Custom Script Workflow

The script runs in five stages:

1. authenticate all worker clients upfront,
2. distribute request batches across threads,
3. execute the target workload (read, write, race, mixed, or failure),
4. collect latencies, successes, and errors per thread,
5. write everything to a timestamped CSV.

### 3.2 What Each Workload Tests

Each workload targets a specific correctness or performance question rather than just generating load:

- **Concurrent Read:** can the system serve many users reading at once without errors or degradation?
- **Concurrent Write:** do simultaneous inserts all go through cleanly without interfering with each other?
- **Race Condition:** does repeated concurrent modification of a single record leave it in a valid state?
- **Mixed Workload:** how does the system behave with reads and writes interleaved, which is closer to real usage?
- **Constraint Violation / Failure Injection:** does the backend reject invalid data and avoid leaving partial writes behind?

## 4 Experimental Setup

All tests ran against the local Flask server at `http://127.0.0.1:5000` with MySQL already running. We used the `user` account for authentication. The parameters for the final run were:

- 100 requests for the concurrent read and mixed workloads, using 16 workers,
- 50 requests for the concurrent write workload, using 8 workers,
- 50 concurrent update attempts on the same ticket for the race condition test, using 16 workers,
- mixed workload split: approximately 70% reads, 30% writes,

- 3 deliberate constraint violations injected in the failure test.

All results reported here are taken from the execution whose output CSV was saved as:

`stress_test_api_20260405_165210.csv`

### 4.1 Metrics Collected

For each workload we recorded average latency (ms), P95 latency (ms), success count, error count, and throughput (rps). For correctness-focused tests we also recorded whether any invalid data was created, whether partial writes survived failures, and explicit atomicity and isolation flags from the script output.

## 5 Experimental Results

Table 1 shows the measured values from the final run. Numbers are taken directly from the CSV and terminal output without modification.

Table 1: Results from `stress_test_api_20260405_165210.csv`

| Test | Avg (ms) | P95 (ms) | Errors | Workers | Throughput | Status |
|---|---|---|---|---|---|---|
| Concurrent Read | 615.73 | 791.27 | 0 | 16 | 24.63 rps | ok |
| Concurrent Write | 39.63 | 53.55 | 0 | 8 | 192.26 rps | ok |
| Race Condition | 0.00 | 0.00 | 0 | 16 | N/A | ok |
| Mixed Workload | 458.30 | 734.90 | 0 | 16 | 32.27 rps | ok |
| Constraint Violation | N/A | N/A | 0 | N/A | N/A | ok |
| Failure During Exec | N/A | N/A | 0 | N/A | N/A | ok |

The additional correctness metrics from the same run:

- concurrent read successes: 100 / 100,
- concurrent write successes: 50 / 50,
- race condition target: ticket ID 1215,
- ticket validity after race: `ok`,
- mixed workload: 69 reads, 31 writes, 0 errors,
- constraint violations attempted: 3, caught: 3, invalid data created: 0,
- atomicity verified: `YES`,
- failure-during-execution scenarios: 3,
- mid-execution failures simulated: 10,

- partial data found after failures: 0,
- isolation verified: `YES`.

*Note: the race condition workload shows 0.00 ms for both average and P95 latency. This is a known artefact of how the script calculates timing for that particular workload and does not reflect actual response time. The correctness output — zero errors and a valid final ticket state — is the meaningful result here.*

# 6 Detailed Workload Analysis

## 6.1 Concurrent Read Testing

We sent 100 GET requests to `/tickets` using 16 threads. Every request succeeded; average latency was 615.73 ms and P95 was 791.27 ms, giving a throughput of 24.63 rps.

The latency is higher than the write test, which is expected — reading tickets involves fetching and serialising a potentially large result set. More importantly, there were no failures at all across 16-way parallel access. The endpoint stayed stable and returned valid data throughout.

## 6.2 Concurrent Write Testing

We sent 50 POST requests to `/tickets` using 8 threads. All 50 completed successfully with an average latency of 39.63 ms, P95 of 53.55 ms, and throughput of 192.26 rps. This was by far the fastest workload.

The low latency makes sense: each insert is a self-contained operation with no dependency on other threads. The key observation is not the speed, but that zero requests failed. Concurrent ticket creation did not produce any duplicate-key errors, deadlocks, or constraint violations.

## 6.3 Race Condition Testing

This was the most important correctness test. We picked a single ticket (ID 1215) and had 16 threads fire 50 concurrent PATCH requests at it. All 50 returned success, and a subsequent GET on that ticket confirmed its fields were consistent and valid.

The 0.00 ms latency values in the table are a script timing artefact, not real measurements. What matters is that after 50 simultaneous updates on one record, the database did not end up with a corrupted or internally inconsistent ticket. InnoDB's row-level locking serialised the conflicting writes correctly.

## 6.4 Mixed Workload Testing

100 requests using 16 workers, roughly split 70/30 between reads and writes. The actual split landed at 69 reads and 31 writes. Zero errors. Average latency was 458.30 ms with P95 at 734.90 ms and throughput at 32.27 rps.

Latency sits between the read-only and write-only results, which is about what you would expect. The P95 being close to 735 ms suggests a small number of requests queued behind others, but nothing timed out or failed. The system handled interleaved reads and writes without any correctness issues.

### 6.5 Constraint Violation and Failure Injection

We deliberately sent three malformed requests:

1. a ticket creation with an invalid `category_id` (non-existent foreign key),
2. a ticket creation with an invalid `location_id` (non-existent foreign key),
3. a ticket creation with an out-of-range priority value.

All three were rejected with HTTP 500 responses. The script confirmed that zero rows were inserted despite the attempts. The atomicity field reported `YES`, meaning the failed transactions left no partial state behind.

This is straightforward InnoDB behaviour — foreign key checks and constraints are enforced at commit time, and any violation rolls back the whole transaction. But it is worth verifying through the API layer because an application bug (e.g., catching the exception and not re-raising) could still allow a partial write. That did not happen here.

### 6.6 Failure During Execution

We ran three additional scenarios to simulate failures that occur mid-operation rather than due to bad input.

**Scenario 1: Client timeout during ticket creation.** We set the client-side timeout to 0.001 seconds, forcing the client to abort before reading the response. The server had already begun handling the request. After the timeout, we checked the database and found a fully-formed ticket with all required fields — the server-side transaction had already committed atomically before the client disconnected.

**Scenario 2: Connection abort during a batch of 5 inserts.** We simulated a crash after the second successful insert by closing the connection mid-way. The database showed exactly 2 of the 5 intended tickets. The remaining 3 were never submitted, so there was nothing to roll back — each HTTP request is its own transaction boundary, so partial batches at the application level do not create partial database state.

**Scenario 3: Concurrent workers with injected failures.** 20 threads attempted concurrent inserts, but we randomly forced 40% of them to fail before completing. Of the 20, 12 succeeded and 8 failed. Crucially, the 8 failures did not corrupt or affect the 12 successful inserts.

Final integrity check after all three scenarios:

- initial ticket count: 1276,
- final ticket count: 1291,

- tickets created during the test: 15,
- partial/invalid records found: 0,
- atomicity: `YES`, isolation: `YES`.

# 7 ACID Analysis

## 7.1 Atomicity

Three different tests speak to atomicity. The constraint violation test showed that all three bad inserts were fully rolled back with no partial data. The failure-during-execution test showed that 10 simulated mid-operation failures left zero partial rows. The concurrent write test showed 50 clean inserts with no half-written records. In all cases the atomicity flag came back `YES`.

## 7.2 Consistency

Foreign key violations were caught before they could corrupt referential integrity. The race condition test left ticket ID 1215 in a fully consistent state after 50 concurrent updates. After the failure injection tests, all 1291 tickets in the database had complete, valid field values. No constraint was broken at any point in the run.

## 7.3 Isolation

The race condition test is the clearest isolation test: 50 threads writing to the same row simultaneously, with no corrupted outcome. The failure injection test adds another angle — the 8 failed concurrent operations did not interfere with the 12 that succeeded. The isolation flag was explicitly reported as `YES`.

## 7.4 Durability

All successful writes went through the full application stack and were committed to InnoDB. The final ticket count of 1291 matched the expected 1276 + 15, with no discrepancy between what the API reported and what was in the database. Since InnoDB uses a write-ahead log and fsync on commit, these writes are durable against process crashes.

# 8 Authentication Overhead Under Load

One thing worth noting is that all test traffic went through JWT-based authentication. Every request in every workload carried a valid token that the server had to verify. We did not bypass this to make the numbers look better. The latency figures in the table therefore include token validation time on top of database access time.

This is arguably the right way to benchmark a production web application — isolating the database from the auth layer would give misleading numbers. The fact that all

workloads completed without auth-related failures also confirms that the token validation path is not a bottleneck under 16-way concurrency.

## 9 Discussion

Looking at the results overall, a few things stand out. First, the zero-error rate across every single workload is the strongest result. We ran 100 reads, 50 writes, 50 same-record updates, and 100 mixed requests — none of them failed. That is a solid baseline for a locally deployed application under test-scale load.

Second, the concurrent write throughput (192.26 rps) is notably higher than the read throughput (24.63 rps). This seems counterintuitive but makes sense given that reads fetch and serialise multiple rows while inserts are point operations on a single new row. The P95 on reads (791 ms) is also higher, which reflects the same effect under the tail of the latency distribution.

Third, the failure injection and constraint violation tests are arguably more informative than the performance numbers. Getting a clean ACID report under synthetic load is useful, but demonstrating that the system stays correct when things go wrong — timeouts, aborted connections, bad input — is what actually matters for a real deployment.

The one anomaly worth flagging again is the 0.00 ms latency on the race condition test. This is a timing measurement bug in our script, not a meaningful result. We are reporting it as-is because the CSV contains that value, but the correctness outcome (zero errors, valid final state) is what we actually verified.

## 10 Conclusion

Module B tested FixIIT under conditions it would realistically face in use: many users reading at once, many users creating tickets simultaneously, two users editing the same record, and various failure modes. The results were consistently clean across all workloads.

Key takeaways from `stress_test_api_20260405_165210.csv`:

- 100 concurrent reads, 0 errors, 24.63 rps at 615.73 ms average,
- 50 concurrent writes, 0 errors, 192.26 rps at 39.63 ms average,
- 50 same-ticket concurrent updates left ticket ID 1215 in a valid state,
- mixed workload (69R + 31W) completed with 0 errors at 32.27 rps,
- all 3 injected constraint violations were caught; 0 invalid rows created,
- 10 simulated mid-execution failures produced 0 partial records,
- atomicity and isolation both explicitly verified as `YES`.

The backend handles concurrent load correctly. Constraint enforcement and transactional rollback work as expected through the API layer. Simulated failures during execution do not corrupt the database. Based on what we measured, the system meets the Module B requirements around multi-user correctness and failure resilience.